Function copy

```
>>> def welcome():
        print ('Welcome to the course')

    wel = welcome
          function name

    del welcome

    print (wel())
```

→ copying function to another identifier

→ deleting original function reference

⇒ Welcome to the course

Closures

⇒ A function inside a function.

⇒ You can pass a function as an arg.

eg

```
>>> def wel(a):
        def welcome():
            print (a)

        return welcome()

    wel ('Hello')
```

⇒ "Hello"

A variable declared in parent function can be used in the nested / sub function.

eg (A) Passing function as a parameter.

```
>>> def a(func):
        def sub_a():
            func('Hello')
        return sub_a()
    a(print)
```

func is print

print('Hello')

⇒ Hello

function print as a parameter.

(A)(A) Decorators

```
@func_name
def new_func():
    pass
```

→ used to pass a function into another pre-defined function as an argument.

eg

```
>>> def a(func):
        func()

    @a
    def say_Hello():
        print('Hello')
```

pre-defined func

@func_name decorator

function to be passed as the arg.

⇒ Hello

Decorator with arguments

```
>>> def repeat(n):
        def decorator(func):
            def wrapper(*args, **kwargs):
                for i in range(n):
                    func(*args, **kwargs)
            return wrapper
        return decorator
>>> @repeat(3)
    def say_hello():
        print('Hello')
>>> say_hello()
```

⇒ Hello
Hello
Hello

? Not understood

return func
VS
return func()

? why.